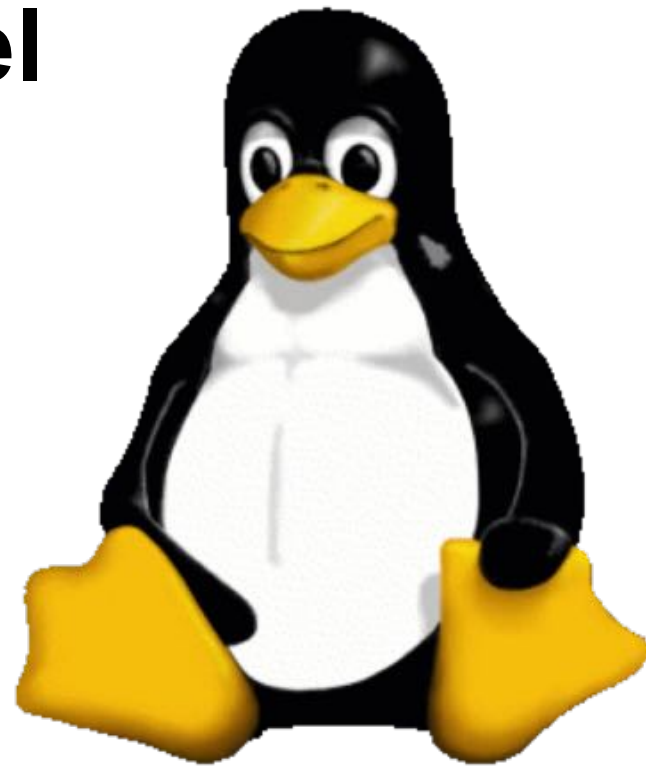


CS353 Linux Kernel

Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn

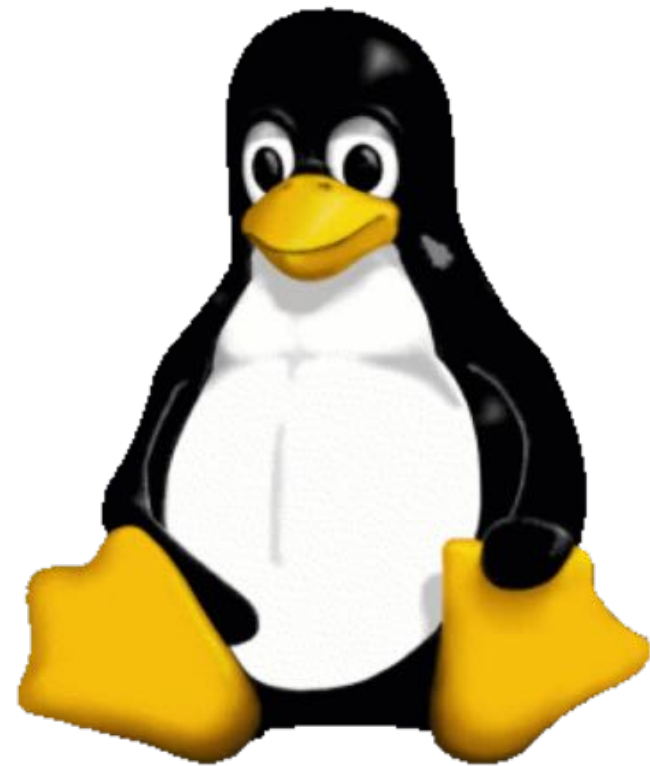


上海交通大学



3A. Process Management -- Introduction

Quan Chen
Dept. of CSE, SJTU
chen-quan@cs.sjtu.edu.cn



上海交通大学

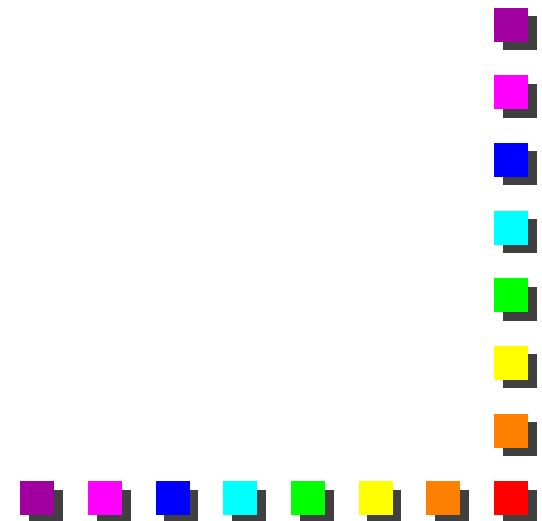


Outline

- Process and Thread
- Process Descriptor
- PID
- Wait Queues
- Creating and Destroying Processes



上海交通大学



Processes and Threads

- *Process*: an instance of a program in execution
 - The unit of resource management
- *(User) Thread*: an execution flow of the process
 - Pthread (POSIX thread) library
 - The unit of program execution
- *Why threads?*
 - Need parallelism
 - Low utilization of single-thread program
 - High overhead of multi-process
 - Downside: Synchronization required.



Kernel Threads

- *A clone of kernel*
 - *Each kernel thread provides a specific function*
 - *Asynchronize IO*
 - Kernel threads run only in kernel mode
 - They use only linear addresses greater than PAGE_OFFSET
- *Low overhead*
 - *Only require kernel stack and registers for context switch*
 - *Multi-threads kernel*

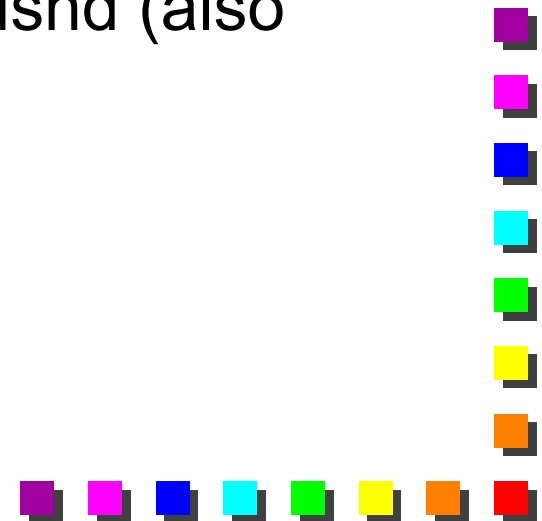


上海交通大学



Kernel Threads

- `kernel_thread()`: to create a kernel thread
- Example kernel threads
 - Process 0 (swapper process), the ancestor of all processes
 - Process 1 (init process)
 - Others: `keventd`, `kapm`, `kswapd`, `kflushd` (also `bdfush`), `kupdated`, `ksoftirqd`, ...



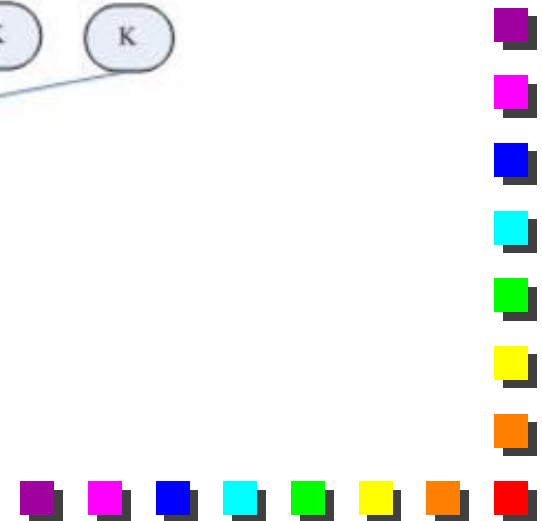
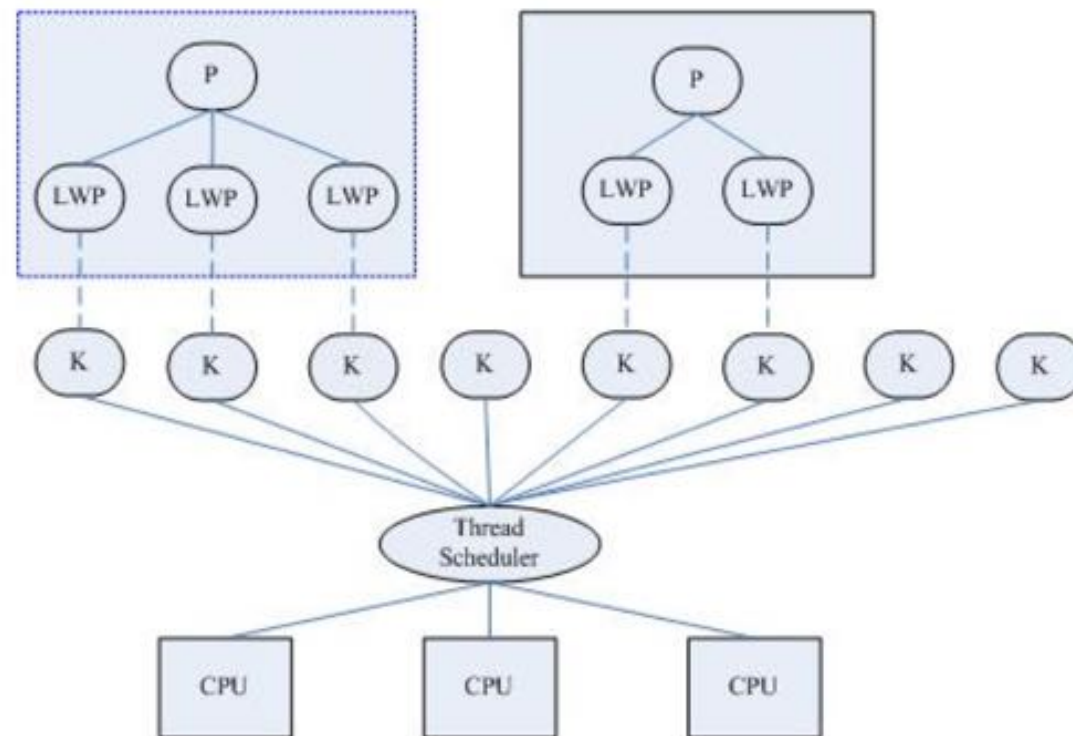
Lightweight Processes

- *Lightweight process (LWP)*: used to offer better support for multithreaded applications
 - Each process may have multiple LWPs
 - Each LWP is associated with a thread
 - LWP may share resources: address space, open files, ...
 - Examples of pthread libraries that use LWP:
 - LinuxThreads
 - Native POSIX Threads Library (NPTL)
 - IBM's Next Generation Posix Threading Package (NGPT)

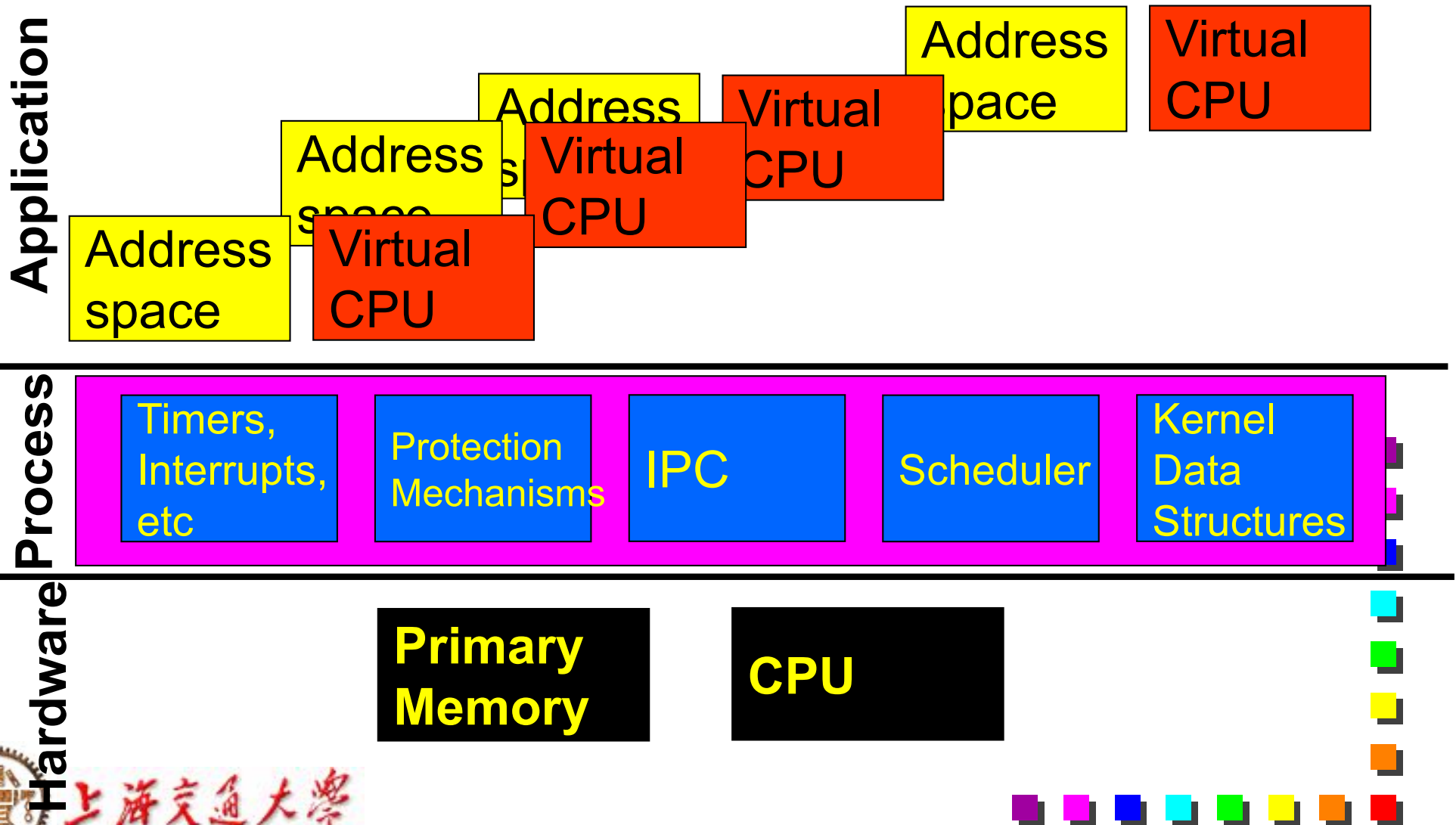


Limitations of LWP

- Require frequent system calls



Process Management



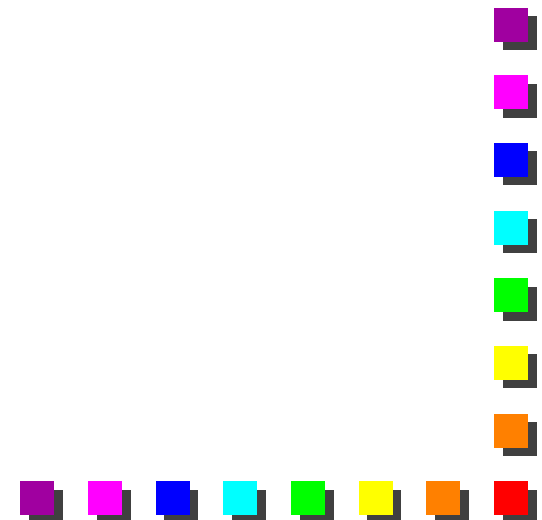
Process Descriptor

■ *task_struct* data structure

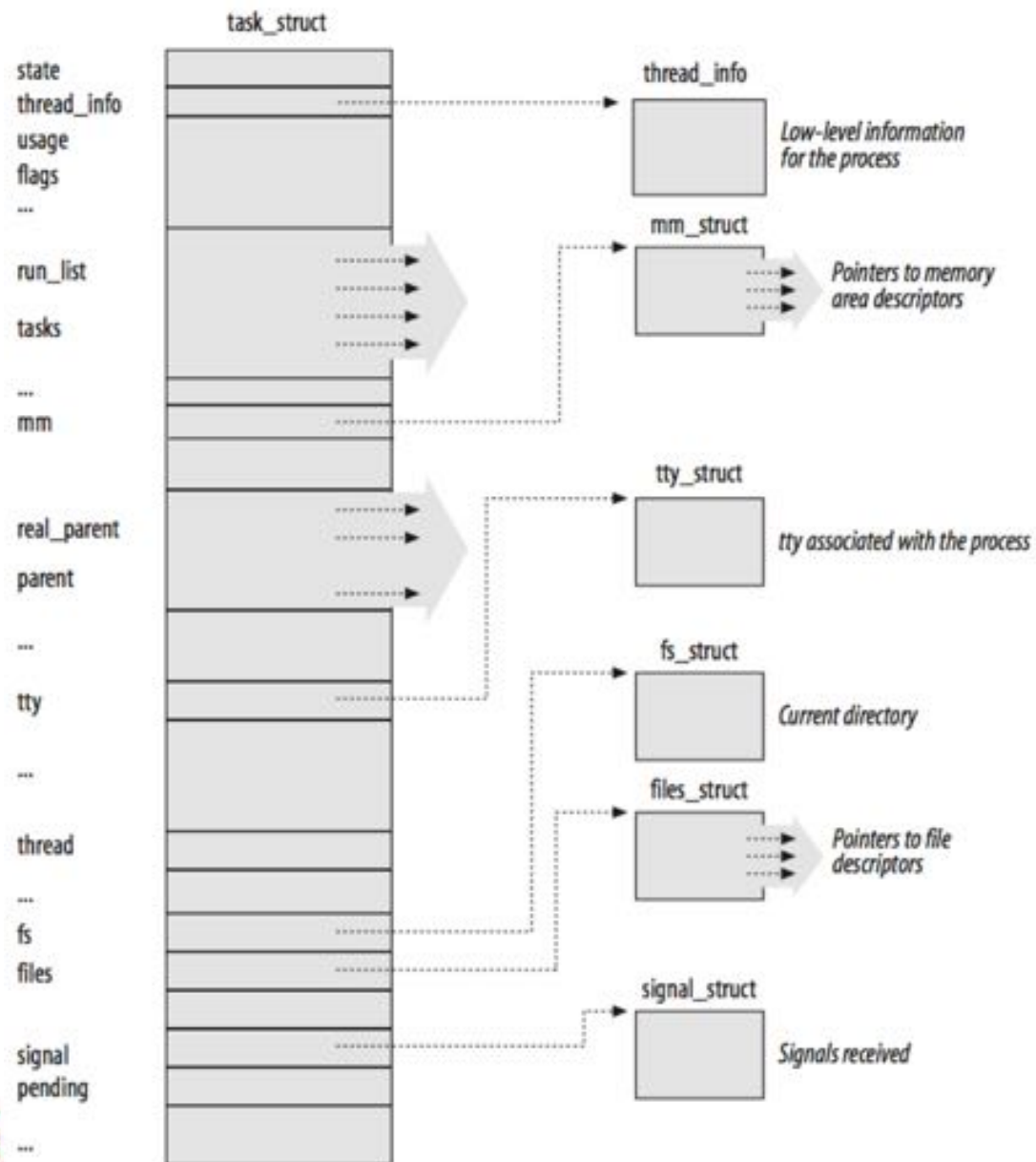
- state: process state
- thread_info: low-level information for the process
- mm: pointers to memory area descriptors
- tty: tty associated with the process
- fs: current directory
- files: pointers to file descriptors
- signal: signals received
- ...



上海交通大学



Linux Process Descriptor

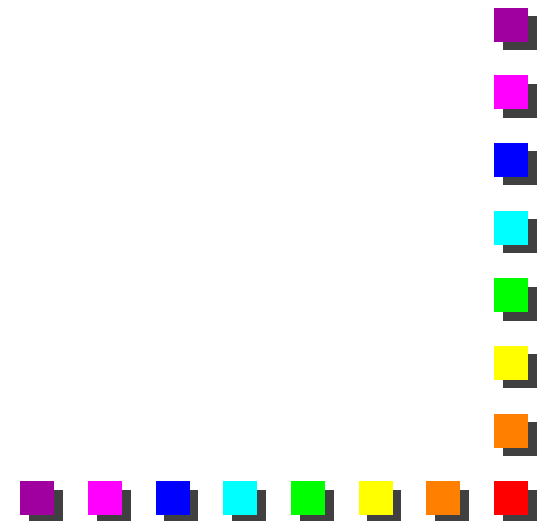


上海交通大学



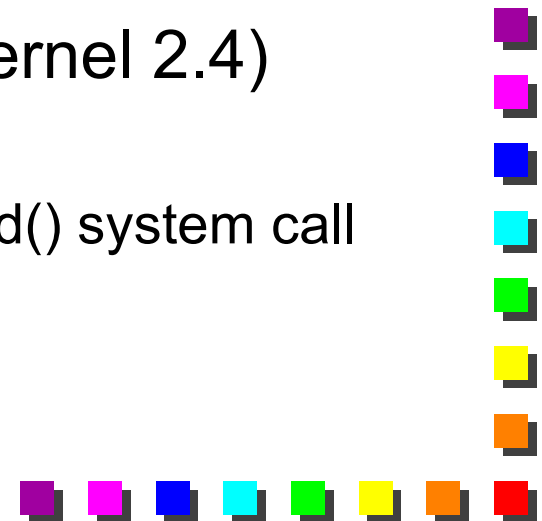
Process State

- TASK_RUNNING: executing
- TASK_INTERRUPTABLE: suspended (sleeping)
- TASK_UNINTERRUPTABLE: (seldom used)
- TASK_STOPPED
- TASK_TRACED
- EXIT_ZOMBIE
- EXIT_DEAD



Identifying a Process

- Process descriptor pointers: 32-bit
- Process ID (PID): 16-bit (~32767 for compatibility)
 - Linux associates different PID with each process or LWP
 - Programmers expect threads in the same group to have a common PID
 - *Thread group*: a collection of LWPs (kernel 2.4)
 - The PID of the first LWP in the group
 - *tgid* field in process descriptor: using getpid() system call



Process Descriptor



上海交通大学

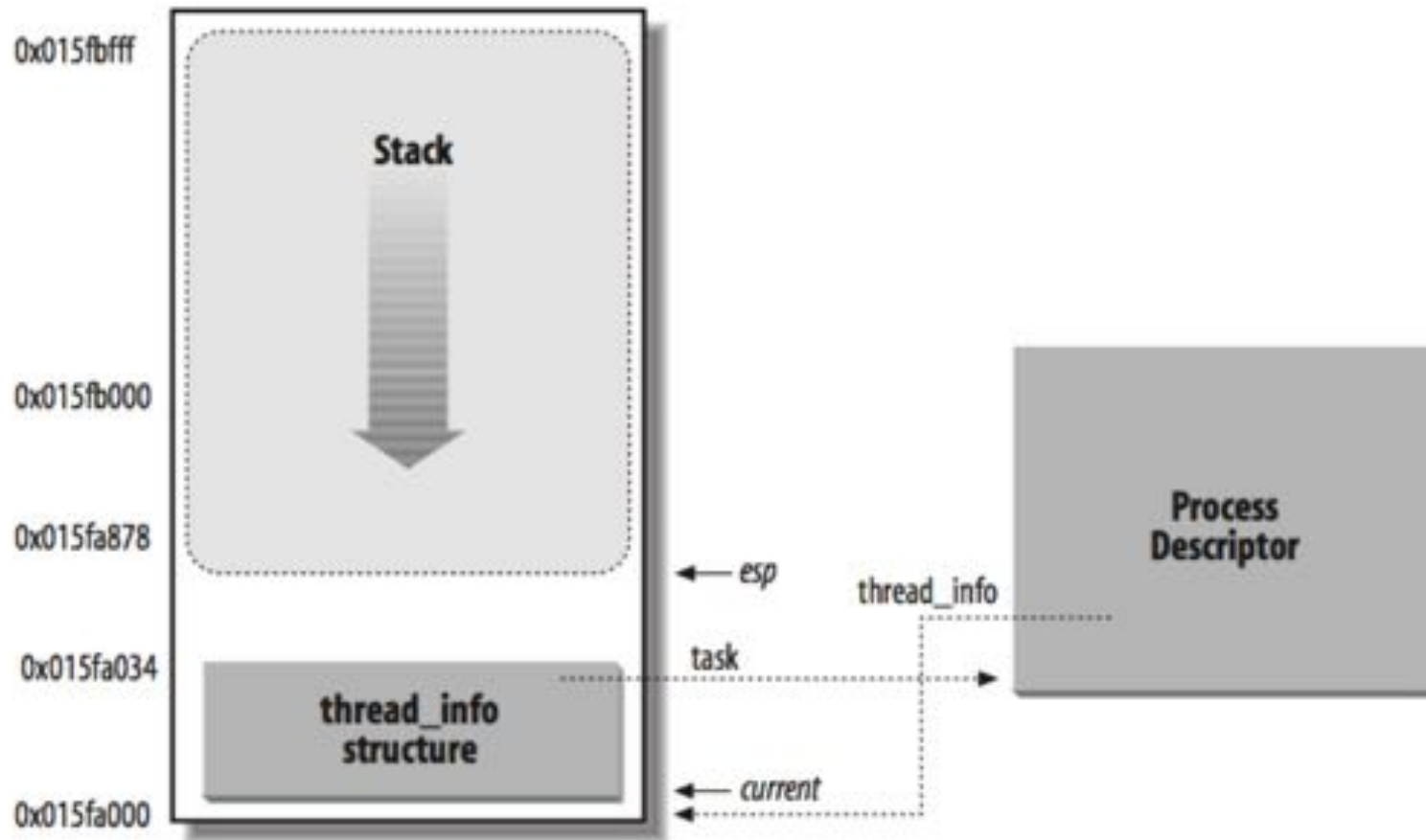


Process Descriptor Handling

- union thread_union {
 struct thread_info thread_info;
 unsigned long stack[2048];
};
- Two data structures in 8KB (2 pages)
 - *thread_info* structure (new to 3rd ed.)
 - Kernel mode process stack



Storing the *thread_info* Structure and the Process Kernel Stack

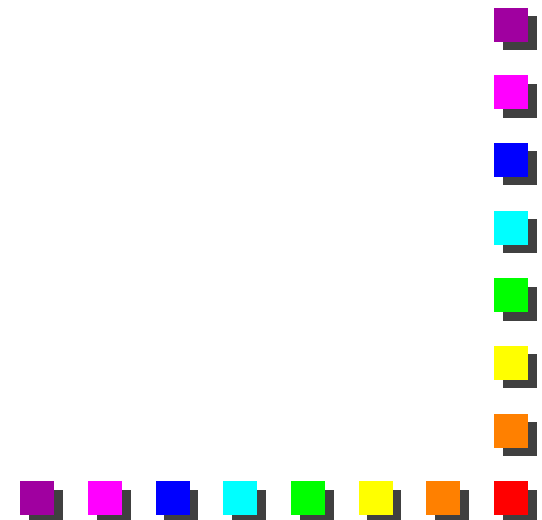


上海交通大学

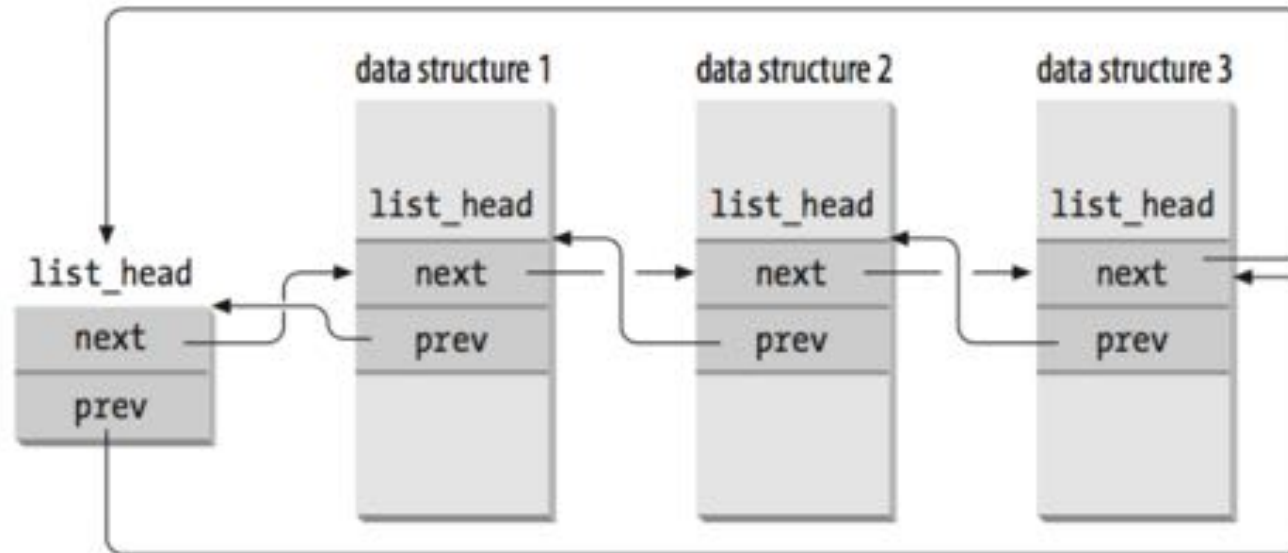


Identifying the Current Process

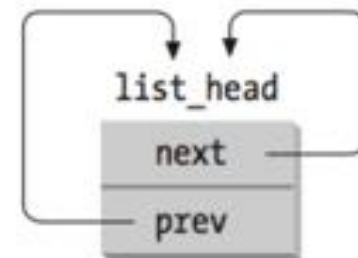
- Obtain the address of *thread_info* structure from the *esp* register
 - `current_thread_info()`
 - `movl $0xffffe000, %ecx`
`andl %esp, %ecx`
`movl %ecx, p`
- The process descriptor pointer of the process currently running on a CPU
 - The *current* macro: equivalent to `current_thread_info()->task`
 - `movl $0xffffe000, %ecx`
`andl %esp, %ecx`
`movl (%ecx), p`



Doubly Linked Lists Built with the *list_head* Data Structure



(a) a doubly linked list with three elements



(b) an empty doubly linked list



上海交通大学

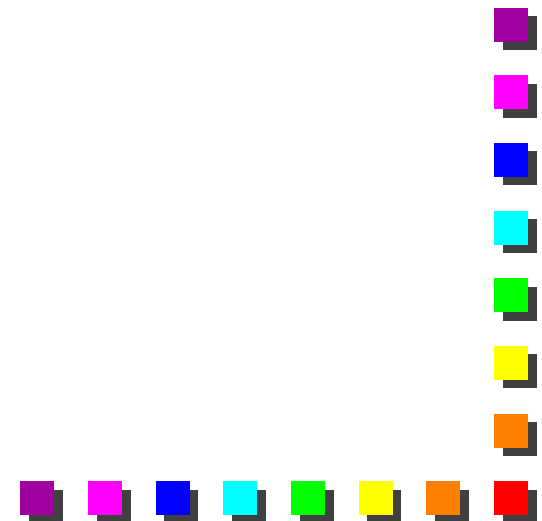


List Handling Functions and Macros

- LIST_HEAD(list_name)
- list_add(n,p)
- list_add_tail(n,p)
- list_del(p)
- list_empty(p)
- list_entry(p,t,m)
- list_for_each(p,h)
- list_for_each_entry(p,h,m)



上海交通大学



The Process List

- *tasks* field in *task_struct* structure
 - type *list_head*
 - *prev*, *next* fields point to the previous and the next *task_struct*
- Process 0 (swapper): *init_task*
- Useful macros:
 - SET_LINKS, REMOVE_LINKS: insert and remove a process descriptor
 - #define for_each_process(p) \for (p=&init_task; (p=list_entry((p)->tasks.next), \struct task_struct, tasks) \) != &init_task;)



List of TASK_RUNNING processes

■ *runqueue*

- *run_list* field in task_struct structure: type list_head

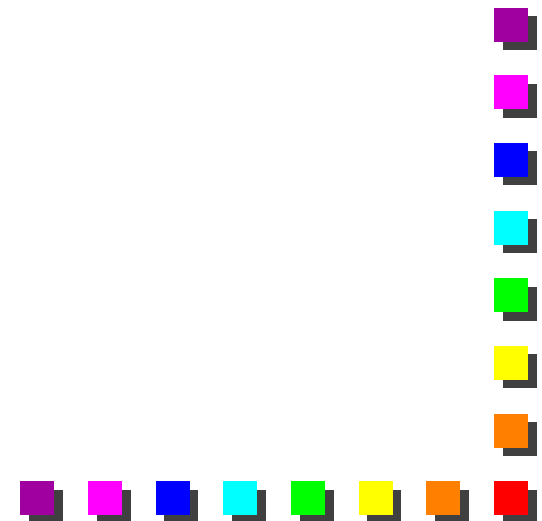
■ Linux 2.6 implements the runqueue differently

- To achieve scheduler speedup, Linux 2.6 splits the runqueue into 140 lists of each priority!
- *array* field of process descriptor: pointer to the *prio_array_t* data structure
 - nr_active: # of process descriptors in the list
 - bitmap: priority bitmap
 - queue: the 140 list_heads
- enqueue_task(p, array), dequeue_task(p, array)

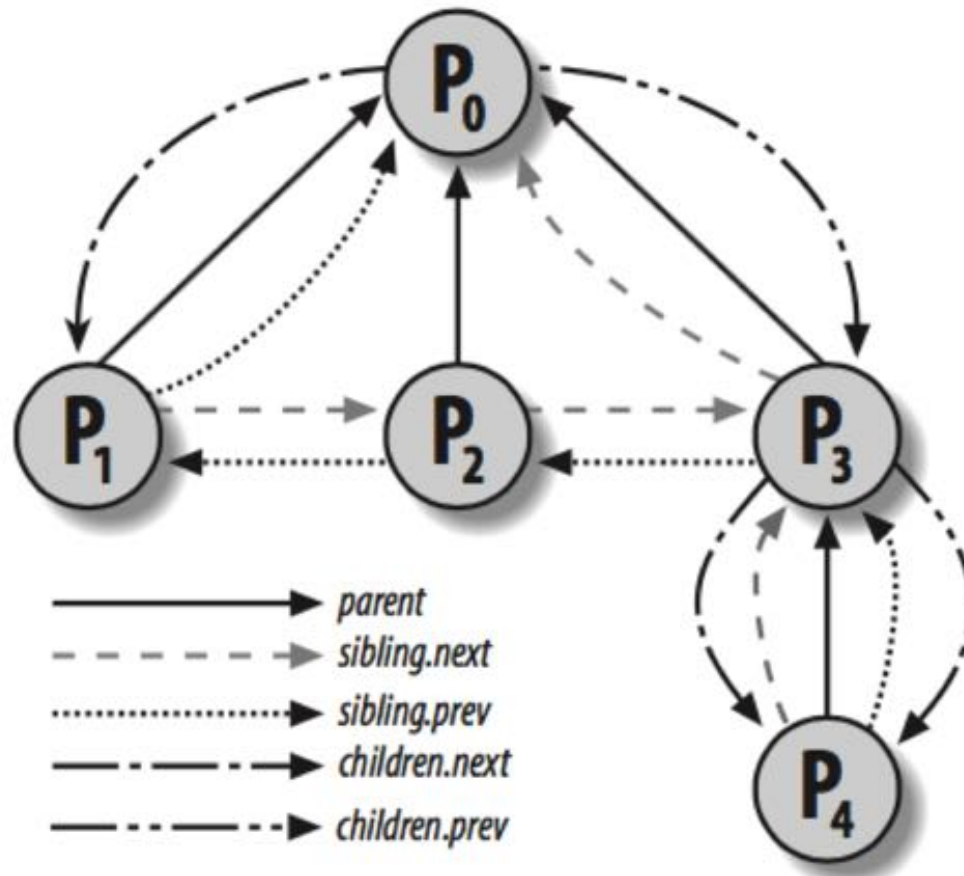


Parenthood Relationships among Processes

- Process 0 and 1: created by the kernel
 - Process 1 (*init*): the ancestor of all processes
- Fields in process descriptor for parenthood relationships
 - `real_parent`
 - `parent`
 - `children`
 - `sibling`



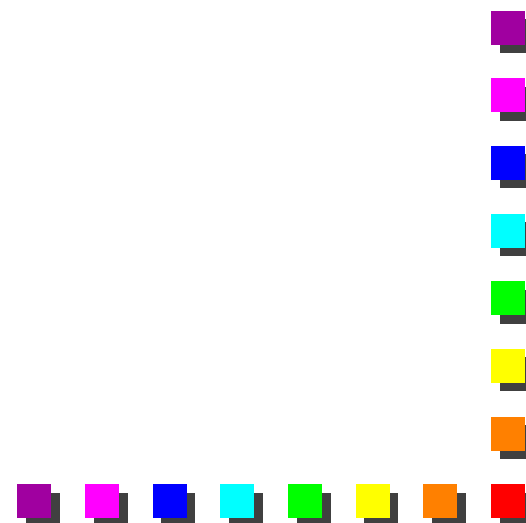
Parenthood Relationships among Five Processes



PID



上海交通大學



Pidhash Table and Chained Lists

- To support the search for the process descriptor of a PID
 - Sequential search in the process list is inefficient
- The *pid_hash* array contains four hash tables and corresponding field in the process descriptor
 - pid: PIDTYPE_PID
 - tgid: PIDTYPE_TGID (thread group leader)
 - pgrp: PIDTYPE_PGID (group leader)
 - session: PIDTYPE_SID (session leader)
- *Chaining* is used to handle PID collisions



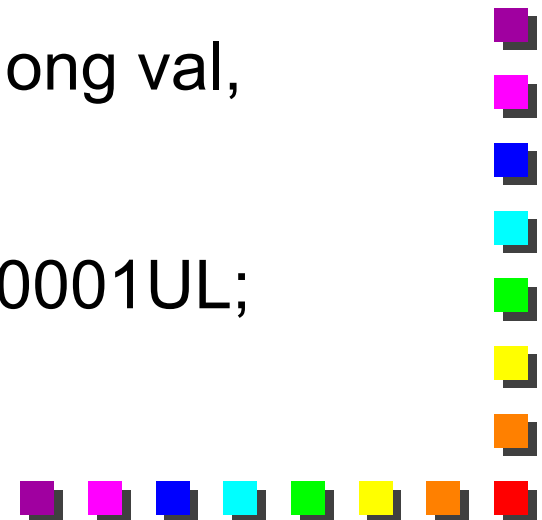
The pidhash Table

- Size of each pidhash table: dependent on the available memory
- PID is transformed into table index using pid_hashfn macro

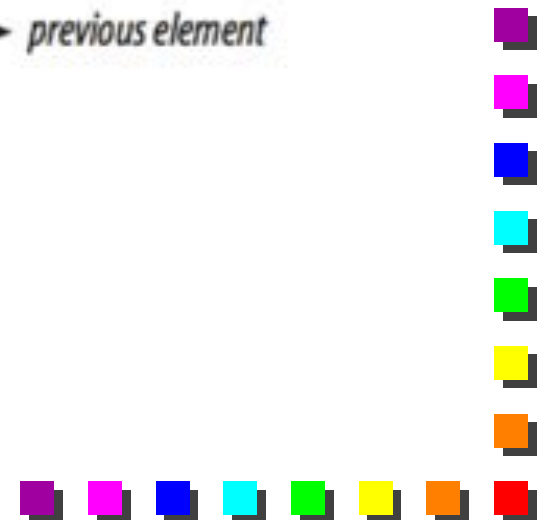
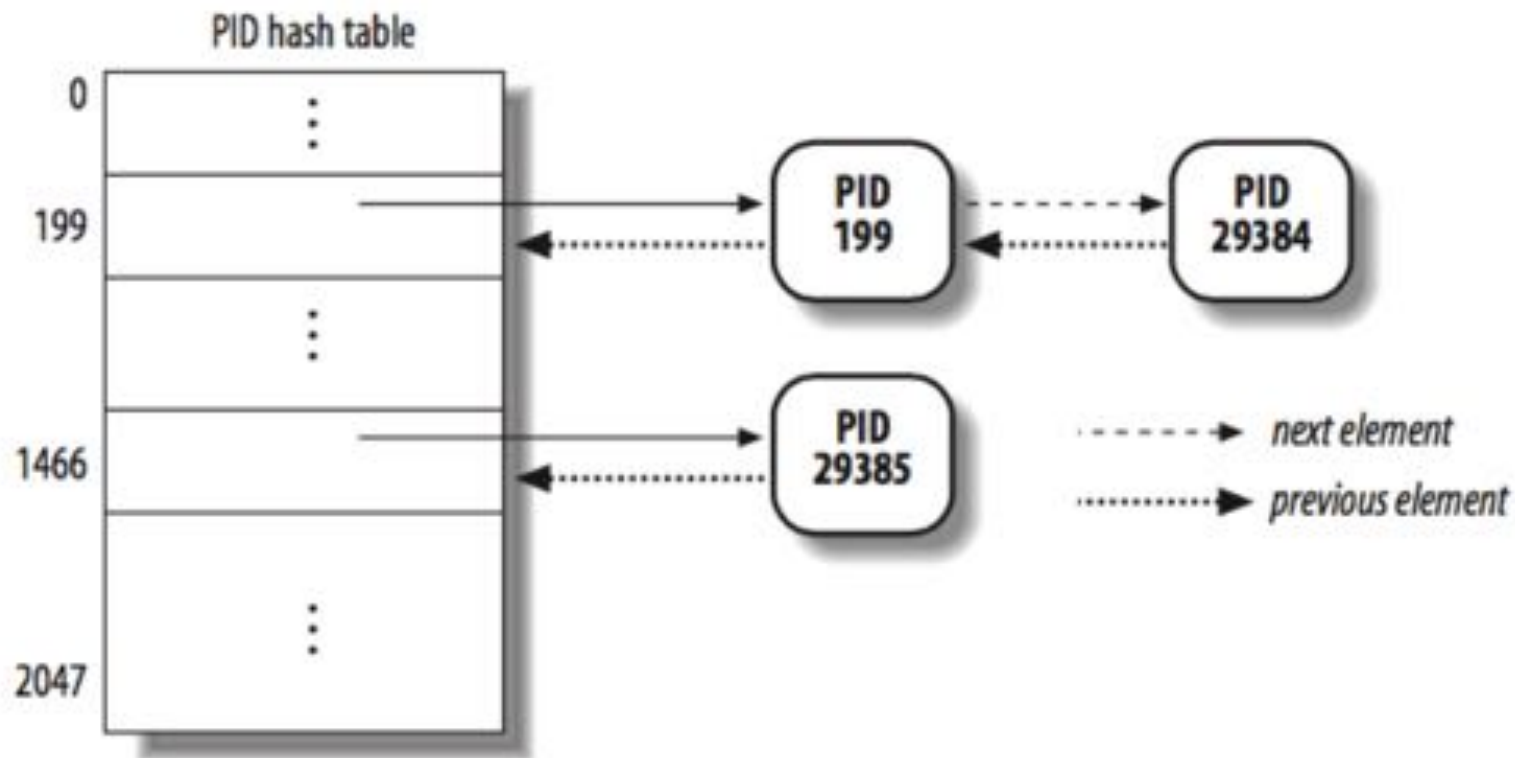
- #define pid_hashfn(x) hash_long((unsigned long)x, pidhash_shift)
- unsigned long hash_long(unsigned long val, unsigned int bits)
 - {
 - unsigned long hash = val * 0x9e370001UL;
 - return hash >> (32-bits);
 - }



上海交通大学



A Simple Example PID Hash Table and Chained Lists



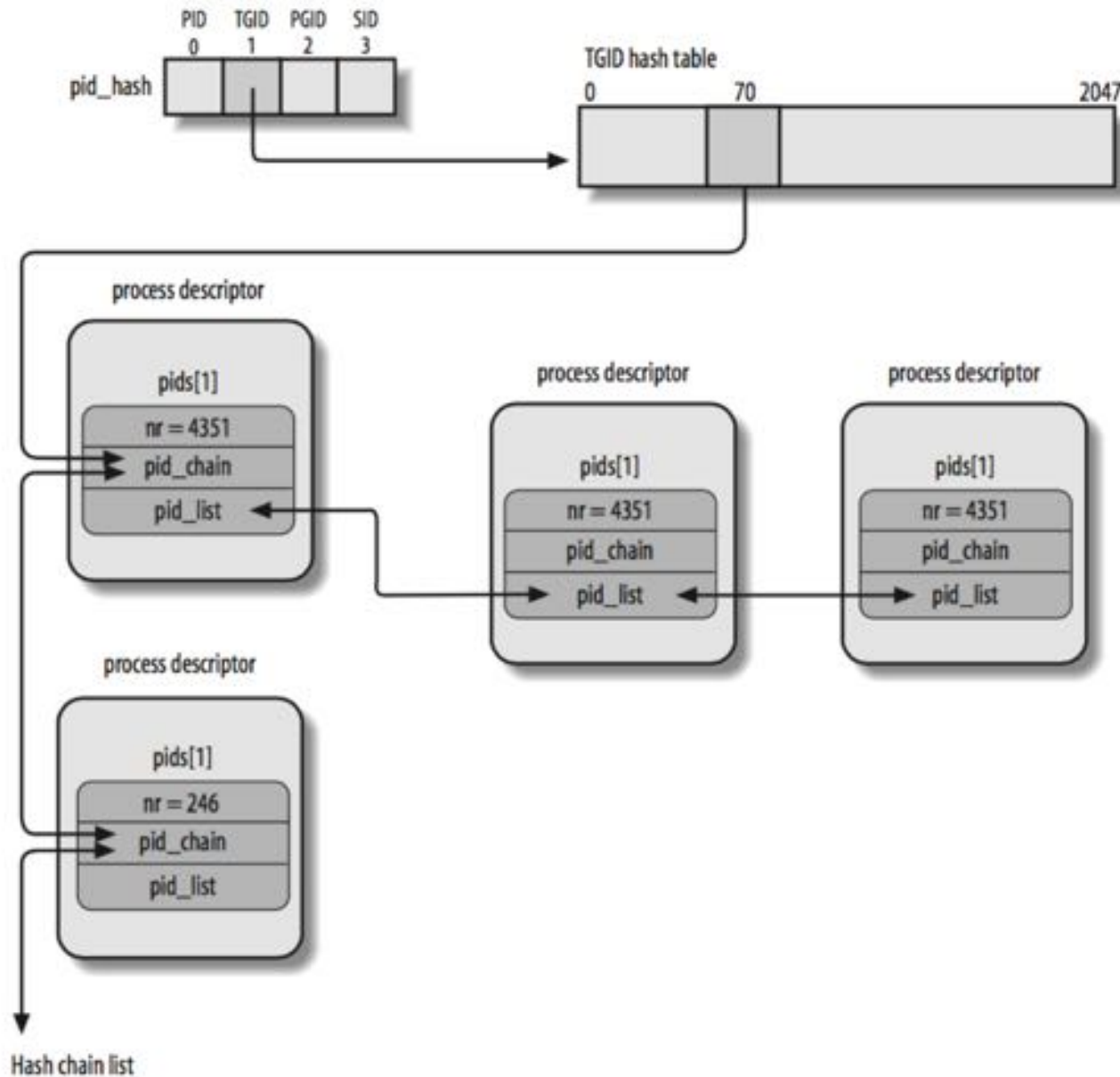
PID

- *pids* field of the process descriptor: the pid data structures
 - nr: PID number
 - pid_chain: links to the previous and the next elements in the hash chain list
 - pid_list: head of the per-PID list (in thread group)

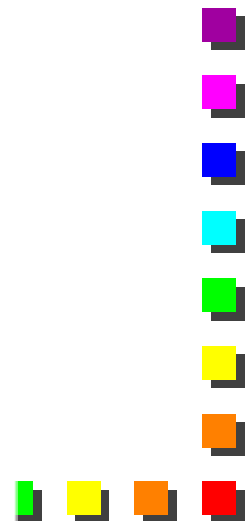


The PID Hash Tables

Maintained
by kernel



上海交通

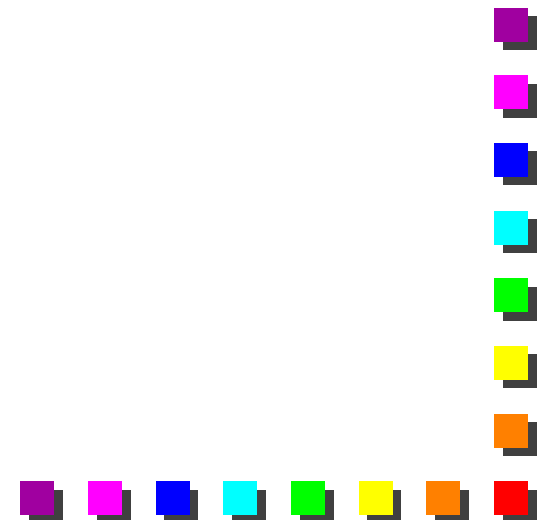


PID Hash Table Handling Functions and Macros

- `do_each_task_pid(nr, type, task)`
- `while_each_task_pid(nr, type, task)`
- `find_task_by_pid_type(type, nr)`
- `find_task_by_pid(nr)`
- `attach_pid(task, type, nr)`
- `detach_pid(task, type)`
- `next_thread(task)`



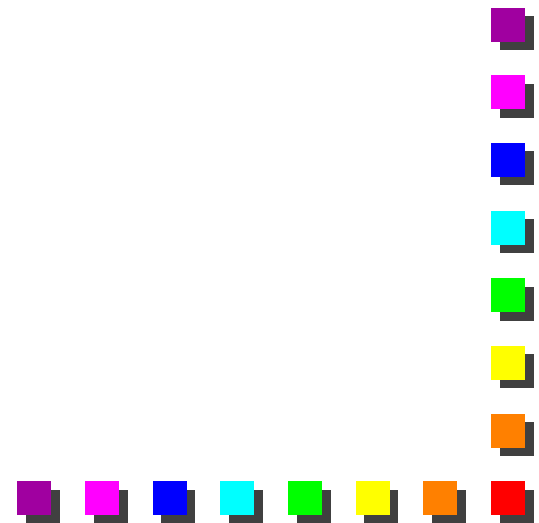
上海交通大学



Wait Queues



上海交通大学

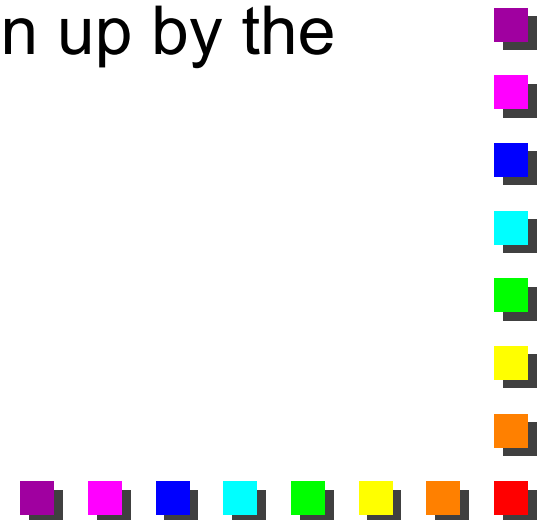


How Processes are Organized

- Processes in TASK_STOPPED, EXIT_ZOMBIE, EXIT_DEAD: not linked in lists
- Processes in TASK_INTERRUPTABLE, TASK_UNINTERRUPTABLE: *wait queues*
- Two kinds of sleeping processes
 - *Exclusive process*
 - *Nonexclusive process*: always woken up by the kernel when the event occurs

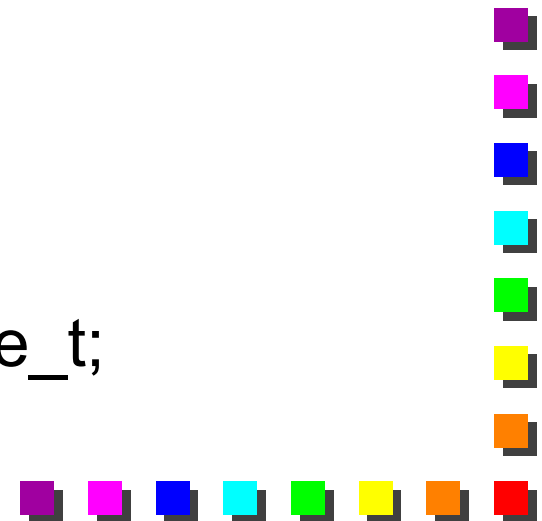


上海交通大学



Wait Queues

- struct __wait_queue_head {
 spinlock_t lock;
 struct list_head task_list;
};
typedef struct __wait_queue_head
wait_queue_head_t;
- struct __wait_queue {
 unsigned int flags;
 struct task_struct * task;
 wait_queue_func_t func;
 struct list_head task_list;
};
typedef struct __wait_queue wait_queue_t;



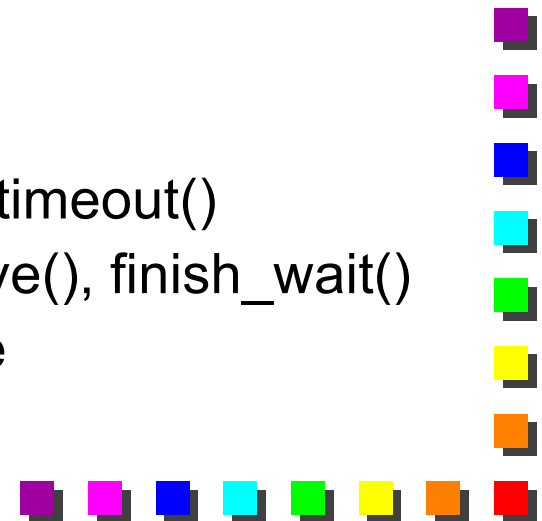
Handling Wait Queues (1)

■ Wait queue handling functions:

- `add_wait_queue()`
- `add_wait_queue_exclusive()`
- `remove_wait_queue()`
- `wait_queue_active()`
- `DECLARE_WAIT_QUEUE_HEAD(name)`
- `init_waitqueue_head()`

■ To wait:

- `sleep_on()`
- `interruptible_sleep_on()`
- `sleep_on_timeout()`, `interruptible_sleep_on_timeout()`
- `Prepare_to_wait()`, `prepare_to_wait_exclusive()`, `finish_wait()`
- Macros: `wait_event`, `wait_event_interruptible`



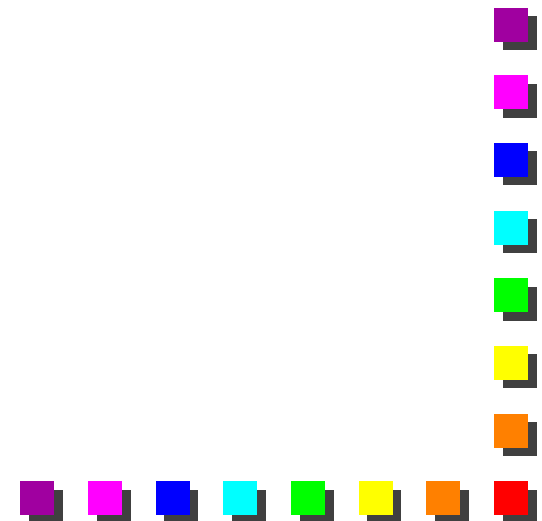
Handling Wait Queues (2)

- To be woken up:

- Wake_up, wake_up_nr, wake_up_all,
wake_up_sync, wake_up_sync_nr,
wake_up_interruptible, wake_up_interruptible_nr,
wake_up_interruptible_all,
wake_up_interruptible_sync,
wake_up_interruptible_sync_nr



上海交通大学

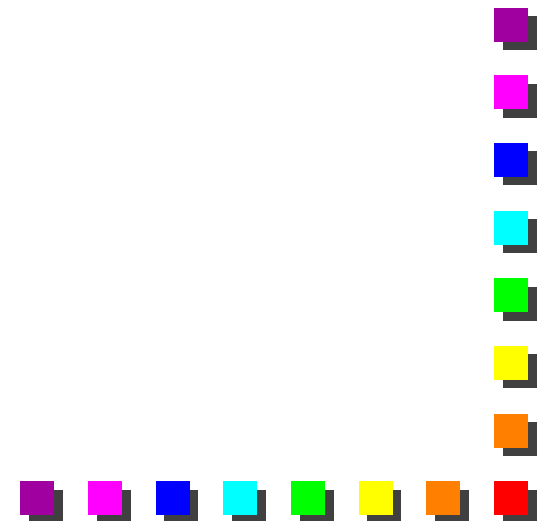


Process Resource Limits

- RLIMIT_AS
- RLIMIT_CORE
- RLIMIT_CPU
- RLIMIT_DATA
- RLIMIT_FSIZE
- RLIMIT_LOCKS
- RLIMIT_MEMLOCK
- RLIMIT_MSGQUEUE
- RLIMIT_NOFILE
- RLIMIT_NPROC
- RLIMIT_RSS
- RLIMIT_SIGPENDING
- RLIMIT_STACK



上海交通大学

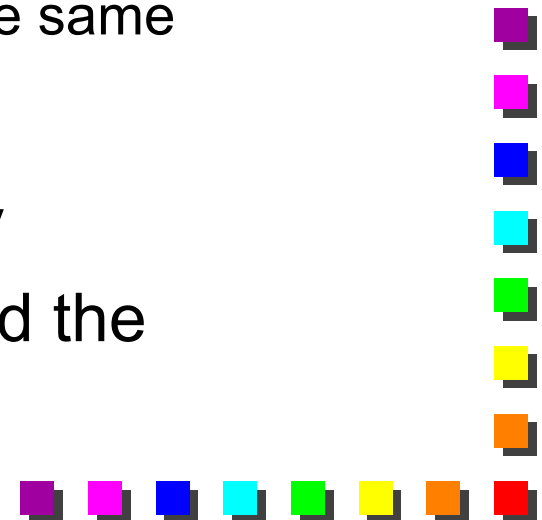


Process Switch

- Process switch, task switch, *context switch*
 - *Hardware* context switch: a far jmp (in older Linux)
 - *Software* context switch: a sequence of mov instructions
 - It allows better control over the validity of data being loaded
 - The amount of time required is about the same
- Performing the Process Switch
 - Switching the Page Global Directory
 - Switching the Kernel mode stack and the hardware context



上海交通大学



Task State Segment

- TSS: a specific segment type in x86 architecture to store hardware contexts



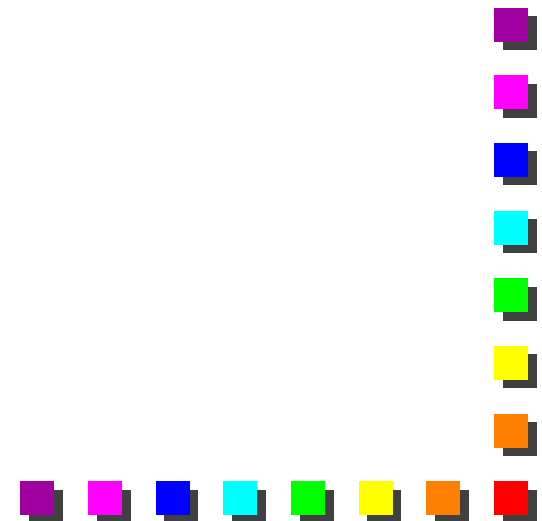
上海交通大学



Creating and Destroying Processes

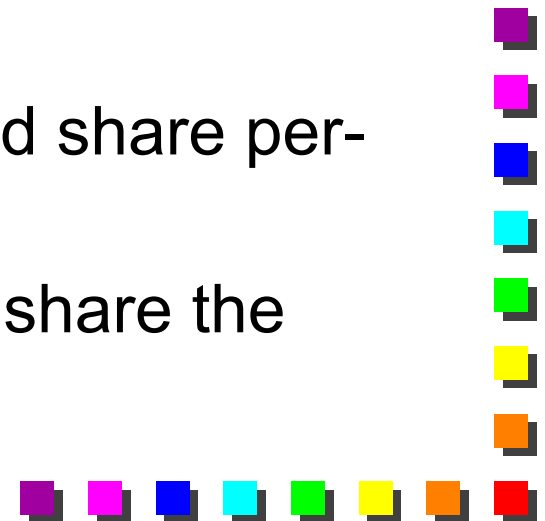


上海交通大学



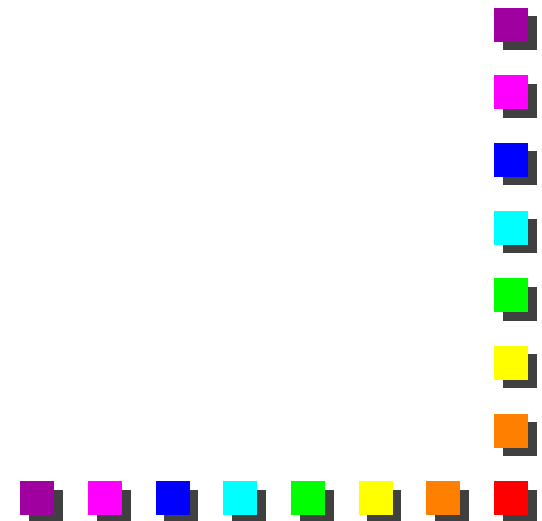
Creating Processes

- In traditional UNIX, resources owned by parent process are duplicated
 - Very slow and inefficient
- Mechanisms to solve this problem
 - **Copy on Write**: parent and child read the same physical pages
 - Lightweight process: parent and child share per-process kernel data structures
 - `vfork()` system call: parent and child share the memory address space



clone(), fork(), and vfork() System Calls

- clone(fn, arg, flags, child_stack, tls, ptid, ctid):
creating lightweight process
 - A wrapper function in C library
 - Uses clone() system call
- fork() and vfork() system calls: implemented by clone() with different parameters
- Each invokes *do_fork()* function



Destroying Processes

- `exit()` library function
 - Two system calls in Linux 2.6
 - `_exit()` system call
 - Handled by `do_exit()` function
 - `exit_group()` system call
 - By `do_group_exit()` function
- Process removal
 - Releasing the process descriptor of a zombie process by `release_task()`



End



上海交通大學

